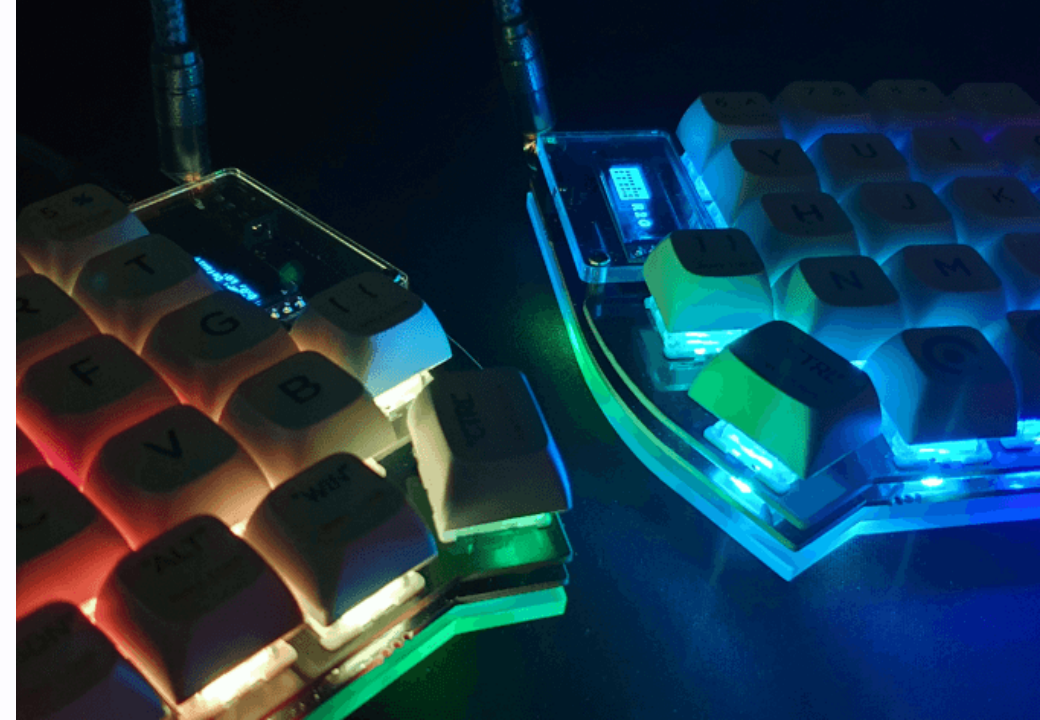


Using HTMX To Make Interactive Elements In Drupal

DrupalCamp England 2026

Philip Norton

- Developer at Code Enigma
- Writer at `#! code`
(www.hashbangcode.com)
- NWDUG co-organiser



Source Code

- Talk is available at:
<https://github.com/hashbangcode/>
or via QR code.
- All code seen can be found
at <http://bit.ly/>
- More resources at
www.hashbangcode.com



Using HTMX To Make Interactive Elements In Drupal

HTMX In Drupal

- HTMX added to Drupal in 11.3 as a core component.
- This is the new standard for ajax requests.
- More on that later!

HTMX

What is HTMX?

- JavaScript framework.
- Gives access to ajax and CSS Transitions.
- Small - 16K (minified and gzipped).
- Uses HTML attributes.
- Plugins extend functionality.
- Platform agnostic.

Installing HTMX

- Download and include the single JavaScript file.

```
<script src="htmx.min.js"></script>
```

- There are CDN options available.

HTMX Attributes

- 11 attributes are commonly used.
- There are another 20-30 that add more functionality.

For example:

```
<button  
  hx-get="index.php"  
  hx-target="#request-output"  
>Submit</button>
```

HTMX Attributes

```
<button  
  hx-get="index.php"  
  hx-target="#request-output"  
>Submit</button>  
<div id="request-output"></div>
```

This example will:

- `hx-get` = Send a "get" request to the `index.php` path.
- `hx-target` = Write the response in the element with the ID "request-output".

HTMX Requests

- All requests from HTMX will have the `hx-request` header.
- Certain attributes will add further headers to the request.

Responding To The Request

Your response should be in plain HTML.

We could respond with

```
<p>Button clicked!</p>
```

Which would be injected into our page like this.

```
<div id="request-output"><p>Button clicked!</p></div>
```

Responder Class In PHP

A simple class to respond to HTMX requests.

```
class Htmx {  
    public static function isHtmxRequest(): bool {  
        return isset($_SERVER['HTTP_HX_REQUEST']) && $_SERVER['HTTP_HX_REQUEST'] === 'true';  
    }  
  
    public static function isGet(): bool {  
        return $_SERVER['REQUEST_METHOD'] === 'GET';  
    }  
}
```

Responding To The Request

We detect the `hx-request` header and the correct HTTP method and respond.

```
if (Htmx::isHtmxRequest() && Htmx::isGet()) {  
    echo '<p>Button clicked at ' . date('r') . ' .</p>';  
}
```

The div now looks like this:

```
<div id="request-output"><p>Button clicked at Sun, 08 Feb 2026 16:00:23 +0000.</p></div>
```

HTTP Verbs

Different HTTP verbs are available through attributes.

- `hx-get` does a "get" request.
- `hx-post` does a "post" request.
- `hx-delete` does a "delete" request
- `hx-patch` does a "patch" request
- `hx-put` does a "put" request

hx-target

- The target element to be swapped.

```
<button hx-get="index.php" hx-target="#request-output">Submit</button>  
<div id="request-output"></div>
```

hx-trigger

- Defines the action that will trigger the request.

```
<div hx-post="index.php" hx-trigger="click">Click me</div>
```

- Can be left out for elements that have default triggers.
 - Eg. `button` elements automatically have a `click` trigger.

hx-trigger Examples

```
<div hx-post="index.php" hx-trigger="click">Click me</div>
```

```
<div hx-post="index.php" hx-trigger="click once">Click me</div>
```

```
<div hx-post="index.php" hx-trigger="revealed"></div>
```

```
<div hx-post="index.php" hx-trigger="load delay:500ms"></div>
```

```
<input hx-get="index.php" hx-trigger="input changed delay:1s" />
```

hx-select

- Select content to swap with the target element from the response.

HTMX:

```
<button hx-get="index.php" hx-select="#response" hx-swap="outerHTML">Click</button>
```

Response:

```
if (Htmx::isHtmxRequest() && Htmx::isGet()) {  
    echo '<p id="response">Button clicked at ' . date('r') . ' .</p>  
<p>Some extra content that won\'t get displayed.</p>';  
}
```

hx-select-oob

- Select Out Of Band
- Do more than one thing in the request.

HTMX:

```
<button hx-get="index.php" hx-select-oob="#request-output">Send Request</button>  
<div id="request-output"></div>
```

Response:

```
if (Htmx::isHtmxRequest() && Htmx::isGet()) {  
    echo 'Request Sent';  
    echo '<div id="request-output">Button clicked at ' . date('r') . ' .</div>';  
}
```

hx-select-oob

- You can comma separate this property to do multiple things.

HTMX:

```
<button hx-get="index.php" hx-select-oob="#div1,#div2">Send Request</button>  
<div id="div1"></div> <div id="div2"></div>
```

Response:

```
if (Htmx::isHtmxRequest() && Htmx::isGet()) {  
    echo 'Request Sent';  
    echo '<div id="div1">Button clicked at:</div>';  
    echo '<div id="div2">' . date('r') . '</div>';  
}
```

hx-swap

- Controls how an element will be swapped into the page.
- Defaults to `innerHTML` - replace the HTML contents.
 - `outerHTML` - Replace the element.
 - `textContent` - Replace contents without parsing as HTML.

hx-swap

- `beforebegin` , `afterbegin` , `beforeend` , `afterend` - Place the response before or after the element and its children.
- `delete` - Delete the target element from the page.
- `none` - Do nothing with the response (but still process out of band items).

hx-swap-oob

- Swap Out Of Band
- Similar to hx-select-oob but the response is responsible.

HTMX:

```
<button hx-get="index.php" hx-swap="none">Send Request</button>  
<div id="div1">...</div>
```

Response:

```
if (Htmx::isHtmxRequest() && Htmx::isGet()) {  
    echo '<div id="div1" hx-swap-oob="true">Button clicked at ' . date('r') . '</div>';  
}
```

CSS Transitions

- HTMX adds classes to elements during different events.
- `htmx-added` is added to new content before it is added to the page.
- We can use these classes to animate what HTMX is doing with CSS transitions.

CSS Transitions

The following will fade in a bit of content inside the `request-output` element as it is loaded into the page.

```
#request-output p {  
    opacity: 1;  
    transition: opacity 1s ease-in;  
}  
#request-output p.htmx-added {  
    opacity: 0;  
}
```

Configuring HTMX

- It is possible to configure HTMX via a number of settings.
- The preferred way to do with is via a metatag.

```
<meta name="htmx-config" content='{ "defaultSwapStyle": "outerHTML" }'>
```

Configuring HTMX

- Configuration options can be set through JavaScript.

```
<script>  
htmx.config.defaultSwapStyle = 'outerHTML';  
</script>
```

examples

- fetch the time
- website counter
- comment form
- form validator
- infinite scroll

HTMX In Drupal

HTMX In Drupal

-

HTMX in Drupal

The standard usage of HTMX in Drupal is to use the `data-hx-` prefix for attributes.

Eg:

```
<button  
  data-hx-get="index.php"  
  data-hx-target="#request-output"  
>Submit</button>  
<div id="request-output"></div>
```

Drupal Libraries

- `core/htmx` - The core HTMX library.
- `core/drupal.htmx` - Additional scripts for Drupal.

Drupal Integration

- The `\Drupal\Core\Htmx` class is used to
- Wrapper around the Htmx libraries and attribute injection.

The Htmx Class

```
$htmx = new Htmx();
```

HTMX Drupal Response

Drupal has the ability to respond to a HTMX response with HTML.

This uses the

`'Drupal\Core\Render>MainContent\HtmxRenderer'` .

Renders the render array as a little HTML document.

HTMX Drupal Response

HtmxRenderer is invoked by

- `_wrapper_format=drupal_htmx` as a query on the incoming request
- The `_htmx_route=true` on the route replying to the request.

HTMX Trait

- `\Drupal\Core\Htmx\HtmxRequestInfoTrait` is used to assist in responding to Htmx requests.
- Detect the request.
- Inspect what headers are attached to the request.

HTMX With Controllers

HTMX With Forms

- The trait 'HtmxRequestInfoTrait` is part of the FormBase class.
- The most important thing to remember with forms is that you get the entire form back in the response, so you need to pull out what you want with a hx-select attribute.
- Also, forms need to be consistent. You can't just throw elements into the form as it the elements need to exist for the form to build and submit correctly.

examples

- load a page via htmx
- cascading date select with HTMX
- block config forms

Example Links

- [Link to website](#)
- [Link to slide](#)

Page without a footer

- Nope, no footer.

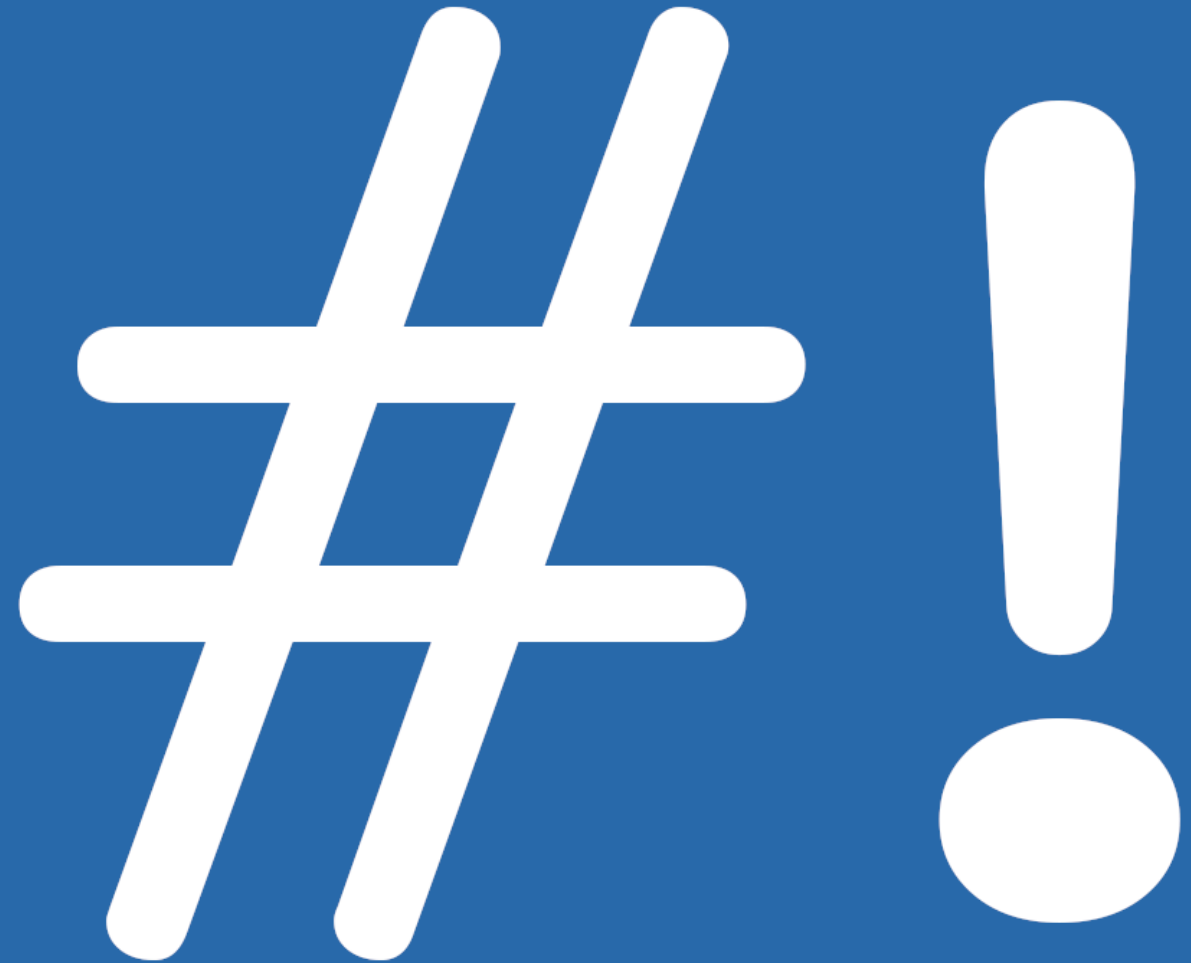
Small Text Slide

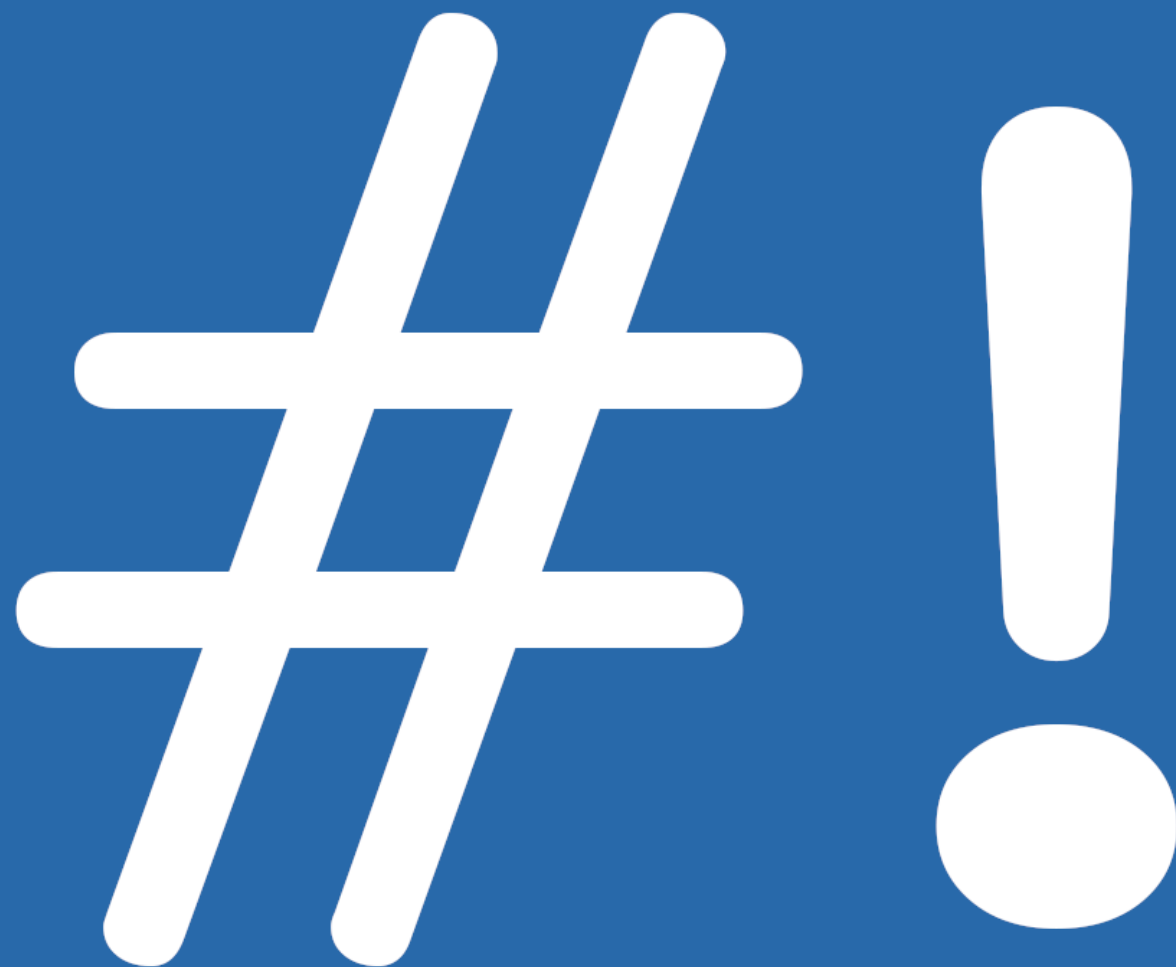
This is small text.

Talk template with image

- Something.

Philip Norton hashbangcode.com
fosstodon.org@hashbangcode
fosstodon.org@philipnorton42





Resources

- htmx.org - <https://htmx.org>
- [HTMX Labs](https://htmxlabs.com) - <https://htmxlabs.com>
- [HTMX Module](https://www.drupal.org/project/htmx) - <https://www.drupal.org/project/htmx>

Questions?

- Slides:

<https://github.com/hashbangcode>
[htmx-talk](#)



Thanks!

- Slides:

<https://github.com/hashbangcode>
[htmx-talk](#)

